DCPT Pipeline Production Deployment Guide

This document provides a comprehensive step-by-step guide for deploying and operating the Degradation Classification Pre-Training (DCPT) pipeline in a production environment. The DCPT framework is designed for robust card valuation and defect classification, leveraging node-specific attention mechanisms and topological data analysis (TDA) for enhanced performance and scientific rigor.

1. Introduction to the DCPT Pipeline

The DCPT pipeline integrates a novel graph neural network architecture with TDA-modulated attention to analyze complex relationships within card data (e.g., image features, metadata) for two primary tasks:

1. **Valuation**: Predicting the market value of collectible cards.
2. **Defect Classification**: Identifying and categorizing various degradation types present on card images.

The core innovation lies in its **node-specific attention mechanism**, which employs unique projection matrices ($W_Q[i], W_K[i], W_V[i], W_O[i], W_{ff}[i]$) for each node, allowing for highly granular feature transformations. Furthermore, the attention coefficients are dynamically adjusted by a sigmoid function of topological descriptors (T_j^*), emphasizing structurally significant regions or features identified through TDA. This approach ensures that the model is sensitive to subtle yet critical topological changes indicative of defects or value-influencing characteristics.

2. Prerequisites

Before deploying the DCPT pipeline, ensure the following prerequisites are met:

2.1 Hardware Requirements

- **GPU**: A CUDA-enabled NVIDIA GPU is highly recommended for efficient training and inference, especially for TDA computations and large datasets. The `tda_gpu_accelerator.py` module (though currently a placeholder) is designed to leverage GPU capabilities.
- **CPU**: Multi-core CPUs are beneficial for data preprocessing and general pipeline orchestration.
- **RAM**: Sufficient RAM to handle dataset loading and model parameters. For large image

datasets, consider at least 32GB or more.

- **Storage**: Ample high-speed storage (SSD/NVMe) for datasets, model checkpoints, and TDA caches.

2.2 Software Requirements

- **Operating System**: Linux (Ubuntu 20.04+ recommended) or Windows Subsystem for Linux (WSL2).
- **Python**: Version 3.8 or higher.
- **CUDA Toolkit**: If using a GPU, ensure the appropriate CUDA Toolkit version is installed and compatible with your PyTorch installation.
- **Git**: For cloning the project repository.

3. Environment Setup

Follow these steps to set up your production environment:

3.1 Clone the Repository

First, clone the DCPT pipeline repository to your production server:

Bash

```
git clone <repository_url>
cd dcpt_pipeline
```

3.2 Create a Virtual Environment

It is best practice to use a virtual environment to manage project dependencies:

Bash

```
python3 -m venv venv
source venv/bin/activate
```

3.3 Install Dependencies

Install all required Python packages. The provided `requirements.txt` (or equivalent `pip install` command) ensures all necessary libraries, including PyTorch, torchvision, scikit-learn, loguru, joblib, opencv-python, pandas, and numpy, are installed.

Bash

```
pip install torch torchvision torchaudio loguru scikit-learn joblib opencv-pyth
```

3.4 Set PYTHONPATH

Add the project root to your `PYTHONPATH` to allow proper module imports:

Bash

```
export PYTHONPATH=$PYTHONPATH:/path/to/your/dcpt_pipeline
```

Replace `/path/to/your/dcpt_pipeline` with the actual absolute path to the `dcpt_pipeline` directory.

4. Data Preparation

The effectiveness of the DCPT pipeline heavily relies on the quality and structure of your input data. While the provided code uses synthetic data for testing, real-world deployment requires careful data handling.

4.1 Data Structure

Your real-world dataset should align with the `CardDataset` expectations, providing:

- **Node Features (`h`)**: Numerical representations of card attributes (e.g., image embeddings, textual features, statistical properties). Shape: `[num_samples, num_nodes, in_features]` .
- **Adjacency Matrix (`adj`)**: Defines the relationships between nodes. Shape: `[num_samples, num_nodes, num_nodes]` .
- **Topological Descriptors (`T_star`)**: Values derived from TDA, representing topological complexity or features for each node. Shape: `[num_samples, num_nodes]` .
- **Prices (`prices`)**: Target values for valuation (e.g., historical sales prices). Shape: `[num_samples]` .
- **Labels (`labels`)**: Target classes for defect classification. Shape: `[num_samples]` .

4.2 Generating TDA Features

Topological descriptors (`T_star`) are crucial for the TDA-modulated attention. You will need to implement a process to extract these from your raw data (e.g., card images). The `tda_optimization/tda_core.py` module provides `TDAApproximator` for fast approximations and `TDACacheManager` for efficient storage of these features. For full TDA computation, external libraries like GUDHI or Ripser-plus-plus might be integrated.

5. Configuration

The `run_pipeline.py` script contains key configuration parameters that should be adjusted for your specific use case and dataset:

Python

```python
# Configuration
num_nodes = 10          # Number of nodes in your graph representation
in_features = 64        # Dimensionality of input node features
hidden_features = 128   # Dimensionality of hidden layers in the model
num_classes = 6         # Number of defect classes
batch_size = 32         # Batch size for training
epochs = 20             # Number of training epochs
device = 'cuda' if torch.cuda.is_available() else 'cpu' # 'cuda' for GPU, 'cpu'
```

Adjust `num_nodes` and `in_features` to match your actual data representation. `num_classes` must correspond to the number of distinct defect categories in your dataset. `batch_size` and `epochs` are hyperparameters that may require tuning for optimal performance.

6. Training the Model

Once the environment is set up and data is prepared, initiate the training process:

Bash

```bash
python3 dcpt_pipeline/run_pipeline.py
```

6.1 Training Logs

The `loguru` library provides detailed logging of the training process, including:

- **Epoch Progress**: Current epoch out of total.
- **Loss Metrics**: Training and validation loss (combined valuation and classification loss).
- **Valuation Metrics**: Mean Absolute Error (MAE) for price prediction.
- **Classification Metrics**: Accuracy for defect classification.
- **Model Checkpointing**: Notifications when the best model (based on validation loss) is saved.

Monitor these logs to track training progress and identify potential issues like overfitting or underfitting.

6.2 Saved Model

Upon completion, the best-performing model (based on validation loss) will be saved to `dcpt_pipeline/models/dcpt_final_model.pth`. This `.pth` file contains the trained model's state dictionary, ready for inference.

7. Model Evaluation and Optimization

7.1 Interpreting Training Metrics

- **Validation Loss**: A decreasing validation loss indicates that the model is learning effectively. If it starts increasing, the model might be overfitting.
- **MAE**: Lower MAE for valuation is better, indicating more accurate price predictions.
- **Accuracy**: Higher accuracy for defect classification is better, indicating correct identification of defects.

7.2 Hyperparameter Tuning

Experiment with different values for `batch_size`, `epochs`, learning rates, and optimizer settings. Techniques like grid search, random search, or more advanced methods (e.g., Bayesian optimization) can be employed.

7.3 TDA Optimization

The `tda_optimization` module is critical for performance:

- **`TDACacheManager`**: Configure the `max_cache_size_gb` to manage disk usage for persistence diagrams and topological signatures. Proper caching prevents redundant and computationally expensive TDA calculations.
- **`TDAApproximator`**: Utilize the `approximation_level` (e.g., 'fast', 'medium', 'accurate') and `sample_ratio` parameters to balance between TDA computation speed and accuracy. The `hybrid_computation` method can dynamically decide between full TDA and approximation based on patch complexity.
- **`TDAGPUAccelerator`**: While currently a placeholder, in a fully integrated system, this module would offload homology computations to the GPU, significantly speeding up TDA feature extraction. Ensure your CUDA environment is correctly configured for this.

7.4 Profiling

For deeper performance analysis, the `tda_profiler.py` (from the original specification) can be adapted to benchmark TDA computation time and memory usage across different

configurations. This helps in identifying bottlenecks and optimizing the TDA pipeline for production throughput.

8. Deployment for Inference

To use the trained model for making predictions in a production setting:

8.1 Load the Trained Model

Python

```python
import torch
from pathlib import Path
from dcpt_pipeline.models.dcpt_model import DCPTModel

# Configuration (must match training configuration)
num_nodes = 10
in_features = 64
hidden_features = 128
num_classes = 6
device = 'cuda' if torch.cuda.is_available() else 'cpu'

model = DCPTModel(
    in_features=in_features,
    hidden_features=hidden_features,
    num_nodes=num_nodes,
    num_classes=num_classes
).to(device)

model_path = Path('dcpt_pipeline/models/dcpt_final_model.pth')
model.load_state_dict(torch.load(model_path, map_location=device))
model.eval() # Set model to evaluation mode
```

8.2 Making Predictions

Prepare your input data (`h` , `adj` , `T_star`) as PyTorch tensors and pass them to the model:

Python

```python
# Example input (replace with your actual data)
# h: [batch_size, num_nodes, in_features]
# adj: [batch_size, num_nodes, num_nodes]
# T_star: [batch_size, num_nodes]

# Ensure inputs are on the correct device
h_input = torch.randn(1, num_nodes, in_features).to(device)
```

```
adj_input = torch.ones(1, num_nodes, num_nodes).to(device)
T_star_input = torch.rand(1, num_nodes).to(device)

with torch.no_grad():
    valuation_pred, classification_pred = model(h_input, adj_input, T_star_inpu

predicted_price = valuation_pred.item()
predicted_class_idx = torch.argmax(classification_pred, dim=1).item()

print(f"Predicted Price: {predicted_price:.2f}")
print(f"Predicted Defect Class Index: {predicted_class_idx}")
```

8.3 Integration

For production use, integrate the inference logic into your existing systems. Common approaches include:

- **REST API**: Wrap the model inference in a web service (e.g., using Flask, FastAPI) to provide real-time predictions.
- **Batch Processing**: For large volumes of data, integrate the model into a batch processing pipeline.
- **Edge Deployment**: For low-latency applications, consider deploying optimized versions of the model directly on edge devices.

9. Maintenance and Monitoring

Continuous maintenance and monitoring are crucial for sustaining model performance in production:

- **Regular Retraining**: Periodically retrain the model with new data to adapt to concept drift and maintain accuracy. Establish a retraining schedule (e.g., weekly, monthly).
- **Performance Monitoring**: Track key metrics (MAE, accuracy, inference latency) in production. Set up alerts for significant performance degradation.
- **Data Drift Detection**: Monitor input data distributions for changes that could impact model performance. If data characteristics shift, retraining or model adjustments may be necessary.
- **Infrastructure Monitoring**: Keep an eye on GPU utilization, memory consumption, and disk I/O to ensure the pipeline runs efficiently.

10. References

- **PyTorch Documentation**: https://pytorch.org/docs/stable/index.html

- **Loguru Documentation**: https://loguru.readthedocs.io/en/stable/
- **Scikit-learn Documentation**: https://scikit-learn.org/stable/documentation.html
- **OpenCV-Python Tutorials**: https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html

---

Author: Manus AI

Date: April 1, 2026